

PRÁCTICA: Preparación y Estandarización de Datos (AI Jobs Dataset)

A continuación, se plantea un ejercicio práctico basado en las técnicas de manipulación de datos. Para esta práctica, utilizaremos como ejemplo un conjunto de datos común en **Kaggle** sobre:

Mercado laboral global de IA y ciencia de datos (2020-2026)

<https://www.kaggle.com/datasets/mann14/global-ai-and-data-science-job-market-20202026>

Descarga el archivo: *ai_jobs.csv* del enlace anterior

Acerca de este archivo

Nombre de la columna	Descripción
id_del_trabajo	Identificador único para cada oferta de empleo
título profesional	Título del puesto original tal como lo indica el empleador
tipo de empresa	Tipo de empresa (Startup, MNC, Laboratorio de Investigación)
industria	Sector industrial de la empresa
país	País donde se encuentra el trabajo
ciudad	Ciudad del lugar de trabajo (se puede anular si es remota)
tipo_remoto	Modalidad de trabajo: remota, híbrida o presencial
nivel de experiencia	Nivel de experiencia requerido (Principiante, Medio, Sénior)
min_años_de_experiencia	Se requieren años mínimos de experiencia
salario_mínimo_usd	Salario mínimo anual en USD
salario_máximo_usd	Salario máximo anual en USD
tipo_de_empleo	Tipo de empleo (Tiempo completo, Contrato)
año_publicado	Año en que se publicó el trabajo (2020-2026)

Contexto: Has obtenido un archivo llamado *ai_jobs.csv* que proyecta la situación laboral en el sector de la inteligencia artificial. Sin embargo, antes de realizar informes estadísticos, debes realizar un proceso de "data cleaning" para asegurar la calidad de la información.

Tareas de la Práctica

1. Importación y Exploración Estructural (RA2.a)

- **Importa** la librería Pandas y carga el archivo usando `read_csv`, especificando que el delimitador es una coma.
- Muestra las **primeras 10 filas** con el método `.head(10)` para comprender la disposición de las columnas y los tipos de datos presentes.
- Utiliza el atributo `.size` y `.index` para conocer el volumen total de registros y etiquetas del eje.

2. Diagnóstico de Calidad y Gestión de Nulos (RA2.b, RA2.c)

- Detecta la presencia de valores ausentes (**NaN**) en todo el DataFrame mediante las funciones `isna()` o `notna()`.

- **Tratamiento de nulos en "ciudad":** Dado que los trabajos remotos pueden no tener ciudad, rellena los valores faltantes en esta columna con la etiqueta **"Sin especificar/Remoto"** usando un valor escalar en fillna().
- **Tratamiento de nulos en "min_años_de_experiencia":** Sustituye los valores nulos por la **mediana** de los años de experiencia para evitar distorsiones en cálculos matemáticos posteriores.

3. Limpieza de Redundancias e Integridad (RA2.c)

- Verifica si existen ofertas de empleo duplicadas basándote en la columna id_del_trabajo usando duplicated().
- Elimina los registros duplicados de forma permanente con el método drop_duplicates().
- Identifica columnas que no aporten valor al análisis estadístico (como id_del_trabajo si ya no es necesario) y elimínala usando el comando del.

4. Transformación y Conversión de Datos (RA2.e, RA2.f)

- **Normalización de Fechas:** Convierte la columna año_publicado al tipo de dato **datetime** de Pandas usando pd.to_datetime() para facilitar futuros análisis temporales.
- **Cálculo de Variables Cuantitativas:** Crea una nueva columna llamada salario_promedio_usd que sea el resultado de la media aritmética entre salario_mínimo_usd y salario_máximo_usd.
- **Conversión de Categóricos:** Identifica las columnas de tipo object (como tamaño de la empresa) y conviértelas al tipo de dato category para optimizar el rendimiento y la memoria.

5. Agrupación por Intervalos (Binning) (RA2.d)

- Utiliza la función pd.cut() para crear una nueva columna llamada rango_salarial. Divide los salarios en tres categorías: **"Entrada"**, **"Competitivo"** y **"Top"**, definiendo manualmente los límites de los intervalos basándote en la columna salario_máximo_usd.

6. Ordenación y Exportación Final

- **Ordena** el DataFrame final de forma descendente por año_publicado y, en caso de empate, por salario_máximo_usd usando sort_values().
- **Exporta** el lote de datos limpio y estandarizado a un archivo llamado ai_jobs_clean.csv empleando el método to_csv, asegurándote de no incluir el índice automático en el archivo final.

NOTAS TEÓRICAS: ¿Cómo se completan los valores nulos?

Según los apuntes de la Unidad, Pandas ofrece diversas estrategias para gestionar los datos faltantes (NaN):

- **Detección:** Se utilizan las funciones isna() (para encontrar nulos) y notna() (para encontrar valores válidos).
- **Eliminación (dropna):** Permite quitar filas o columnas completas. Se puede configurar para eliminar la fila si tiene *algún* nulo, si *todos* sus valores son nulos, o si supera un umbral (*thresh*) de datos faltantes.

- **Rellenado (fillna):** Es la técnica principal para completar datos sin eliminarlos:
 - **Valor escalar:** Sustituir todos los nulos por un número (ej. 0) o un texto (ej. "Sin datos").
 - **Diccionario:** Permite asignar valores de relleno diferentes para cada columna.
 - **Interpolación/Propagación:** Métodos como `bfill()` (relleno hacia atrás) permiten copiar el valor siguiente para completar el hueco, con la opción de limitar cuántos registros consecutivos se rellenan.
 - **Estadísticos:** Es posible rellenar nulos usando el resultado de otras funciones, como la **media** (`mean()`) de la propia columna.